

Clovis SFEIR

+33 6 95 83 95 34 · clovis.sfeir.cs@gmail.com · github.com/sfeirc · linkedin.com/in/clovis-sfeir-388936240 · cloooooo.com · French (Native) – English (C1)

Profile

IMT Atlantique engineering student specialising in performance-critical systems and quantitative software for markets and industrial infrastructure. Built five C++20 systems benchmarked against industry references or verified against labeled ground-truth: ITCH 5.0 feed handler at **97 ns p99** (**8.2×** Aquis Exchange), options engine at **302×** QuantLib, an Ornstein-Uhlenbeck-driven crude-spread trading engine, and a Modbus ICS intrusion detector with **100% attack recall**.

Education

IMT Atlantique – Engineering Degree in Software Engineering (Apprenticeship) *Sep 2025 – Jun 2028*
Nantes, France *Algorithms & Data Structures, OS, Computer Networks, Distributed Systems, Cloud Computing, AI*
Bachelor in Computer Science *Sep 2022 – Jun 2025*
La Rochelle, France *Class rank: 1/85 | Software Engineering, Databases, Cybersecurity*
HRT Macro Placement Challenge 2026 (Hudson River Trading/Partcl) — **24th/124** (top 20%); hybrid analytical placer: force-directed global placement with timing-aware legalization; proxy cost **1.095** beats SA baseline (2.125) and RePlAce (1.458); field includes Stanford, Princeton, ETH Zürich.

Projects

ITCH 5.0 Market Data Feed Handler *C++20, AF_XDP, AVX2, lock-free*

- AF_XDP/BPF steers ITCH 5.0 multicast to a hugepage UMEM ring (kernel bypass); busy-poll receiver on an isolated core, SCHED_FIFO prio 99; AVX2 `_mm256_cmpeq_epi64` level search; open-addressed order map ($\leq 50\%$ load).
- Add **97 ns**, cancel **13 ns** (**8.2×** Aquis Exchange); standard CI: 315/46 ns; sustained **15.7 M msgs/s** at p99=128 ns over 10 s; 0 dropped packets; zero allocations.

Options Pricing Engine *C++20, AVX2, SABR, Crank-Nicolson FD*

- AVX2 Black-Scholes (4-wide `_mm256d`), minimax normal CDF avoids `libm erf()`; Greeks in one pass; **150.9 M prices/s** (**302×** QuantLib); SABR surface (Hagan 2002 + Antonov), L-M calibration, 5×9 surface in 174 μ s.
- American options via Crank-Nicolson FD (sinh-grid, Thomas TDMA): 110 μ s/option; Sobol quasi-MC (antithetic + CV + IS); 10 M-config validation vs QuantLib, max error $< 10^{-6}$.

Crude Oil Spread Trading Engine *C++20, stochastic processes, order book*

- Models refining margins (3:2:1 crack spread, generalized N:M:K) and mean-reverting inter-crude spreads via an Ornstein-Uhlenbeck process (Euler-Maruyama) with a rolling z-score signal, matched by a price-time-priority order book.
- OU stationary distribution verified against theory (500K-step sim); crack-spread formula checked to 10^{-9} ; order-book submit at **720 ns** (non-crossing)/**1.24 μ s** (crossing); 58 assertions/21 tests, ASan+UBSan clean.

Limit Order Book & Matching Engine *C++20, STL, CMake*

- $O(1)$ cancel via `unordered_map`; intrusive list per price level in a pre-pooled `Order` (2 M nodes, 64 MB); 32-byte aligned structs cut cache misses **73%**; SPSC ring decouples I/O from matching.
- Replay of 10 M events: P50 0.6–1.0 μ s, P99 2.3 μ s, P99.9 4.8 μ s; zero hot-path allocations.

ICS Intrusion Detection (Modbus/SCADA) *C++20, protocol parsing, explainable rules*

- Rule-based detector for Modbus TCP (no auth/encryption by protocol design) mapped to documented ICS attacks: protected-register writes (Stuxnet-style setpoint tampering), unauthorized function codes/unit IDs (Industroyer-style rogue commands), write-flood (replay/fuzzing) – every alert names the exact rule that fired.
- Parses MBAP+PDU at **39.2 ns** (**25.5 M msg/s**); parse+detect 50.3 ns; **100% recall, 0 false positives** on a labeled synthetic trace (78 records, 9 attacks); 57 assertions/17 tests, ASan+UBSan clean.

Work Experience

Infotel *Nantes, France*
AI Engineer Apprentice – Lab IA *Sep 2025 – Jun 2028*

- Built a Rust-based agentic orchestration prototype (specs $\rightarrow$ blueprints $\rightarrow$ vertical slices $\rightarrow$ artifacts $\rightarrow$ tests $\rightarrow$ security reports); ownership model prevents use-after-free in concurrent task graphs; quality gates separate build/test/security scans from artifact-only checks; 928 Rust tests; incremental Merkle indexing for $O(\text{changed-files})$ re-index with requirement $\rightarrow$ code $\rightarrow$ test traceability.

Excelia Group *La Rochelle, France*
Software Developer Intern / AI Software Engineer Intern *Two internships: Jan–Mar 2025 & May–Jul 2024*

- RAG pipeline (chunking, OpenAI embeddings, flat FAISS, cross-encoder re-ranking): **40% reduction** in lookup time vs. keyword-search; cut p95 latency 30–40% via embedding cache + async parallelisation; Prometheus/Grafana, Docker, GCP.
- Real-time interview simulator (Next.js/React, GPT-4o, ElevenLabs TTS): streaming responses, scenario-branching state machine, A/B prompt-scoring harness.

Technical Skills

Languages	C++20, Python, Rust, TypeScript, SQL
Perf / Systems	AF_XDP, SIMD/AVX2, lock-free SPSC, cache-aware layout, SCHED_FIFO, acquire/release atomics, ICS/SCADA protocol parsing (Modbus TCP)
ML / Quant	Black-Scholes/SABR, Sobol quasi-MC, Kalman filter, Ornstein-Uhlenbeck, Johansen cointegration, LightGBM/XGB/CatBoost, FAISS
Infrastructure Frameworks	Docker, Prometheus/Grafana, GitHub Actions CI, GCP, Redis, ZeroMQ Flask, Node.js/Express, React/Next.js, REST/gRPC